



SCNNTraffic: A lightweight and energy-efficient traffic classification method based on spiking convolution neural networks

Qin Zeng¹ · Dan Qu^{1,2} · Hao Zhang¹ · Yaqi Chen¹ · Wenlin Zhang^{1,2}

Received: 17 July 2025 / Accepted: 12 February 2026 / Published online: 25 February 2026
© The Author(s) 2026

Abstract

Encrypted traffic classification aims to extract effective representations from encrypted network data, whose content remains opaque, to identify applications or user behaviors. Existing methods mainly use pre-trained models for classification, which increases accuracy but requires high computational resources and is difficult to deploy at the edge. This paper introduces an encrypted traffic classification model utilizing Spiking Convolutional Neural Networks (SCNNs) called SCNNTraffic. SCNNTraffic employs SCNNs to capture the time-varying characteristics of network traffic, facilitating smooth and stable feature extraction. This approach achieves effective classification while significantly reducing energy consumption, highlighting its advantages in resource-constrained environments. Moreover, we introduce a cross-gating module based on spiking neurons that facilitates feature fusion and further decreases power usage. Our experimental results demonstrate that the proposed model significantly reduces both the number of network parameters and energy consumption, achieving accuracy of 98.65% and 92.51% on the ISCX-VPN and Non-VPN datasets, respectively.

Keywords Encrypted traffic classification · Spiking neural networks · User behavior

1 Introduction

In recent years, encrypted traffic technologies have been widely adopted, with the global proportion of encrypted traffic exceeding 90% [1]. Virtual Private Network (VPN) and The Onion Router (Tor) have gained widespread adoption among users by offering enhanced privacy protection and secure connections. While these technologies provide

internet users with greater privacy and anonymity, they also present challenges for cybersecurity regulation and network traffic management.

Encrypted traffic classification is a critical technology in cybersecurity regulation and network traffic management, essential for identifying the types of traffic generated by various applications or web services [2, 3]. However, encrypted traffic technologies have rendered traditional classification methods ineffective, with Deep Packet Inspection (DPI). Methods based on machine learning have achieved some success in classifying encrypted traffic by extracting statistical features and applying models such as Random Forest [4], but they are limited by their dependence on manually designed features. Conversely, methods based on deep learning [5] eliminate the need for manual feature extraction by automatically learning from complex traffic data, thereby achieving notable classification performance. Nonetheless, these methods demand substantial computational resources for both training and inference, potentially impacting normal network operations when deployed on resource-constrained devices.

Currently, the peak data rate of 5G networks has reached up to 10 Gbit/s, with projections indicating that future 6G networks could achieve peak rates of 100 Gbit/s

✉ Dan Qu
qudan_xd@163.com

Qin Zeng
zengqin2361@163.com

Hao Zhang
haozhang0126@163.com

Yaqi Chen
chyqi163@163.com

Wenlin Zhang
wenlinzzz@163.com

¹ Information Engineering University, ZhengZhou 450001, China

² Laboratory for Advanced Computing and Intelligence Engineering, WuXi 214000, China

[6]. A single 64-channel 5G base station can consume up to 3300W of power [7], highlighting the significant energy demands associated with network infrastructure. Network devices must now handle not only vast communication tasks but also cope with escalating energy consumption. Therefore, encrypted traffic classification models should prioritize minimal overhead and energy efficiency, rendering large-parameter models unsuitable for real-time applications and high-speed networks. Lightweight models, such as MATEC [8] and FastTraffic [9], have made strides in reducing model parameters by minimizing inputs and simplifying architectures, but this often results in diminished performance.

Spiking Neural Networks (SNNs) are well-suited for encrypted traffic classification due to their event-driven sparsity and temporal feature capture capabilities, aligning with the demand for lightweight edge deployment. Inspired by this, we propose SCNNTraffic, which separates packet headers and payloads for targeted processing, and introduces a spiking cross-gating module to balance classification accuracy and energy efficiency.

The key contributions of this paper are as follows:

- We propose an encrypted traffic classification model based on SCNNs, achieving a lightweight design that leverages the intermittent firing characteristics of spiking neurons to significantly reduce operational power consumption.
- The comparative experiments we conducted on the ISCX dataset show that SCNNTraffic achieved accuracies of 98.65%, 92.51%, and 98.38% on the ISCX-VPN, NonVPN, and NonTor datasets, respectively, establishing a new state-of-the-art (SOTA) benchmark.
- The t-SNE visualization experiments reveal that spiking neurons enhance the embedding space distances between different classes when applied to payload encoding vectors. This finding underscores the superior capability of SCNNs in processing temporally dynamic encrypted traffic data, which in turn improves classification accuracy and robustness.

2 Related work

2.1 Encrypted traffic classification

Due to the encrypted traffic's characteristic of being opaque to plaintext, traditional deep packet inspection methods based on plaintext have become ineffective. Currently, the primary approaches for analyzing encrypted traffic involve machine learning and deep learning techniques.

Machine learning methods Machine learning techniques for encrypted traffic classification involve extracting statistical features from the data to train models that can infer patterns and classify traffic. For instance, AppScanner [4] utilizes the statistical features of packet sizes to train a Random Forest classifier, while BIND [10] builds upon this approach by incorporating one-dimensional temporal features. However, their performance is constrained by the subjectivity of feature engineering, making it difficult to adapt to the dynamic evolution of encryption protocols. To address the tension between data privacy and model collaboration, federated learning (FL) frameworks [11–13] have been introduced to the traffic classification domain. For instance, Adapt-LFA [14] employs gradient-based optimization to strategically flip training labels, aiming to maximize classification error. While these advances provide theoretical foundations for cross-device collaborative modeling, existing solutions still face the challenge of insufficient extraction of temporal features from encrypted traffic.

Deep learning methods Deep learning methods have emerged as a prevalent trend in encrypted traffic classification due to their ability to automatically learn features without manual design. However, the choice of deep learning method depends on how the traffic data is represented. Inspired by its success in computer vision, deep learning has been adapted for traffic classification tasks. For example, Wang et al. [15] transformed traffic data into grayscale images and utilized Convolutional Neural Networks (CNNs) for classification. Meanwhile, Zhao et al. [16] introduced the YaTC method, which represents traffic as customized MFR matrices processed by Transformer for classification. While these approaches using image or matrix representations have achieved initial successes, they can introduce additional noise into feature extraction by unnecessarily correlating adjacent packet bytes along the vertical axis.

Subsequently, researchers have explored graph-based representations for traffic classification. TFE-GNN [17] represents data as byte graphs with original bytes as nodes, while DE-GNN [18] builds traffic interaction graphs using header and payload vectors as nodes. Although this graph structure introduces new technical framework for traffic classification, it can be constrained by information loss during the graph construction process.

The advancement of Natural Language Processing (NLP) technologies has spurred progress in text-based network traffic classification. ET-BERT, for instance, extracts BURST information from network flows and achieves favorable outcomes through a two-stage adjustment of pre-training and fine-tuning using the BERT model in a textual format. However, this method imposes significant demands

on computational resources and power consumption, posing challenges for deployment in practical network equipment.

In contrast, our proposed method employs a Spiking Convolutional Encoder to downsample and encode text-form inputs, leveraging a Spiking Cross-Gating Module for feature fusion. By utilizing spiking neurons, we achieve effective classification while significantly reducing parameters and energy consumption, making it feasible for deployment on edge devices.

2.2 Spiking neural networks

As Artificial Intelligence(AI) and big data technologies advance, traditional computers based on the von Neumann architecture increasingly struggle to meet exponentially growing computational demands. Spiking Neural Networks offer a promising avenue toward high-efficiency, low-latency computing [19]. By integrating spiking-based computation with neuromorphic hardware, SNNs unlock significant potential for energy-efficient AI [20].

In recent years, SNNs have gained widespread application across multiple domains owing to their energy-efficient properties. In computer vision[21, 22], speech and signal processing[23, 24], SNNs have shown remarkable capabilities. Notably, in NLP, SNNs have demonstrated competitive performance in classification tasks. For instance, Zhang et al. [25] developed efficient text classification using Convolutional Residual Spiking Neural Networks. Moreover, SNNs have achieved success in generative tasks, as exemplified by SpikeGPT [26], which employs spiking neurons to construct a generative language model, thereby substantially decreasing both computational requirements and power usage.

Inspired by these advancements, this study incorporates spiking neurons, leveraging their inherent characteristics—such as event-driven processing and sparse temporal coding—to improve model performance and reduce power consumption.

3 Method

As depicted in Fig. 1, SCNNTraffic framework primarily comprises four key components: a convolutional encoder, a spiking convolutional encoder, a cross-gating unit based on spiking neurons, and a classifier. In the following sections, this paper will provide a detailed introduction to the model's principles and functionalities.

3.1 Traffic representation

Network traffic is composed of one-dimensional byte streams, where bytes, packets, sessions, and entire flows correspond to characters, words, sentences, and documents in NLP, respectively. Therefore, we adopt a text-based format as the input representation for packets. Encrypted network traffic exhibits significant variability depending on the applications, protocols, and services involved, which complicates feature learning. Moreover, network traffic includes many service packets unrelated to classification tasks, such as ARP and TCP handshake packets. These packets are typically unencrypted and difficult to categorize, thereby introducing considerable noise into the classification process.

To address these challenges, we represent traffic using raw bytes and segment the original data files based on the five-tuple (destination IP, source IP, destination port, source

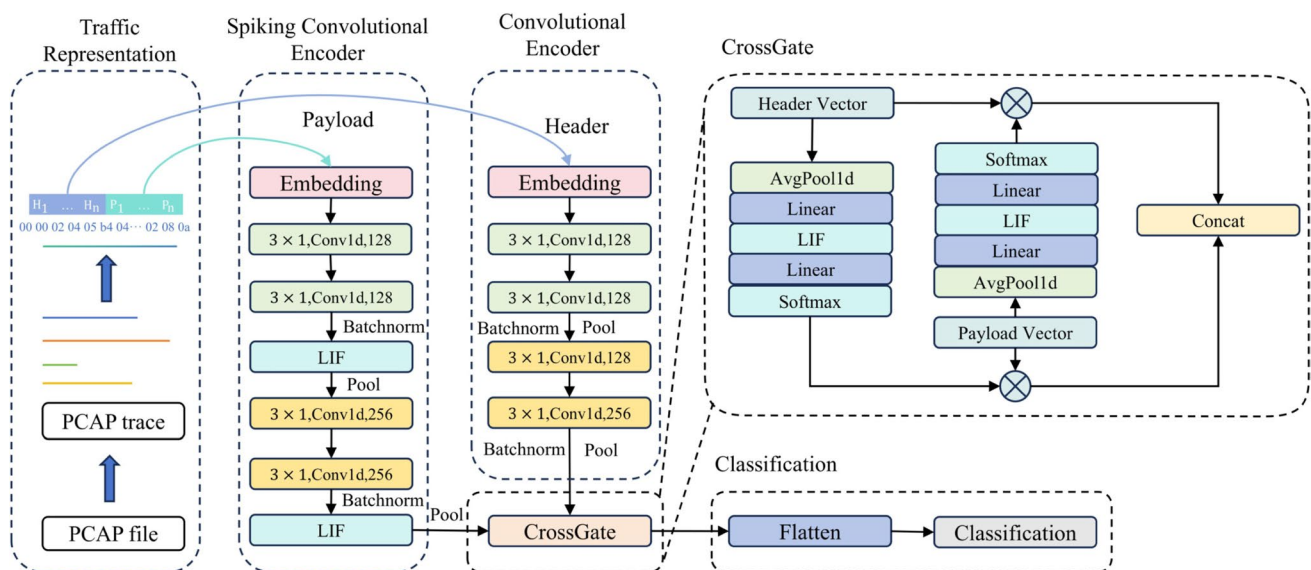


Fig. 1 SCNNTraffic model architecture

port, protocol number). Packets with identical five-tuples are grouped into individual network flows, and classification is performed on a per-flow basis. Each network flow consists of multiple packets that share the same five-tuple, indicating mutual communication between endpoints. As illustrated in Fig. 1, the header and payload of the i -th packet are denoted as H_i and P_i , respectively. We extract the headers and payloads from all packets and concatenate them separately to form a header vector $(H_1, H_2, \dots, H_n)$ and a payload vector $(P_1, P_2, \dots, P_n)$, which are then used as inputs to the model. In encoding method, we adopted a simple integer embedding approach, converting each byte into a vector representation.

3.2 Convolutional encoder

The header of network packets contains essential metadata, including network protocols, packet sizes, timestamps, and other critical information, structured in a fixed format that often includes redundant data. A convolutional encoder acts as a filter to distill key features from this structured information. Utilizing One-Dimensional Convolutional Neural Networks (1D CNNs) not only facilitates feature extraction but also reduces the overall number of network parameters, thereby enhancing computational efficiency. Consequently, a convolutional encoder is employed to encode the header vectors.

The convolutional encoder can be expressed by the following equation:

$$\text{encoder}_h(h) = \text{MaxPool}(\text{Batchnorm}(\text{Conv1d}(h))) \quad (1)$$

where $h \in R^{d_2 \times d_1}$ is the embedded header vector, and $\text{encoder}_h(h)$ is the output of the packet header vector through the convolutional encoder, $L \times d$ are the byte length and embedding dimension, and Conv1d is composed of two 3×1 1D CNNs.

3.3 Spiking Convolutional Encoder

The payload consists of randomized, encrypted data that exhibits significant inter-byte correlations. Since spiking neurons are particularly adept at handling time-varying sequences and can produce more spikes in response to payload data with substantial variations, this paper employs a spiking convolutional encoder for the downsampling and encoding of payload data.

3.3.1 Spiking neurons

The spiking neuron model serves as a mathematical representation of biological neuronal behavior. In this study, we

adopt the Leaky Integrate-and-Fire (LIF) neuron [27], commonly used in computational neuroscience, as our default spiking neuron. The LIF operational model is represented as follows:

$$V[t] = H[t] + \beta(X[t] - (H[t-1] - V_{reset})) \quad (2)$$

$$S[t] = \Theta(V[t] - V_{threshold}) \quad (3)$$

$$H[t] = V[t](1 - S[t]) \quad (4)$$

The three equations above model the processes of charging, firing, and recovery of the neuronal membrane potential. Specifically, V denotes the membrane potential (or hidden state), β is the decay factor, X is the input to the spiking neuron, Θ is the step function, and H represents the reset mechanism following a spike.

Neurons in the LIF model emit spikes only when the accumulated potential reaches the threshold, they remain largely inactive otherwise. This event-driven operational mode significantly reduces unnecessary computations and signal transmissions, leading to lower overall power consumption. Unlike traditional neural networks that operate continuously, SNNs remain idle when no spikes are generated, thus drastically reducing computational requirements.

3.3.2 Architecture of the spiking convolutional encoder

For encrypted traffic data, SNNs built upon LIF neurons are particularly effective at processing such data due to their inherent temporal dynamics. The LIF neuron model operates analogously to a resistor-capacitor circuit, where the membrane potential continuously integrates input current and simultaneously leaks over time. This core mechanism, governed by a time constant τ , allows the neuron to function as a low-pass filter, smoothing out rapid fluctuations in the input. When this potential surpasses a specific threshold, the neuron emits a spike and resets its potential. This process enables the output spike train to encode both the sequence and intensity of network events, capturing essential temporal features directly from the encrypted payload.

Architecturally, as illustrated in Fig. 2, the pulse convolutional encoder integrates spiking neurons with a one-dimensional Convolutional Neural Network (1D-CNN). The processing pipeline begins with the raw payload undergoing preliminary feature extraction via the 1D-CNN. The extracted features are then fed into LIF neurons. These neurons process the input over time, and upon firing, they generate spike trains that encode the temporal dynamics of the features. Finally, a pooling layer downsamples these output spikes to produce a condensed and informative representation.

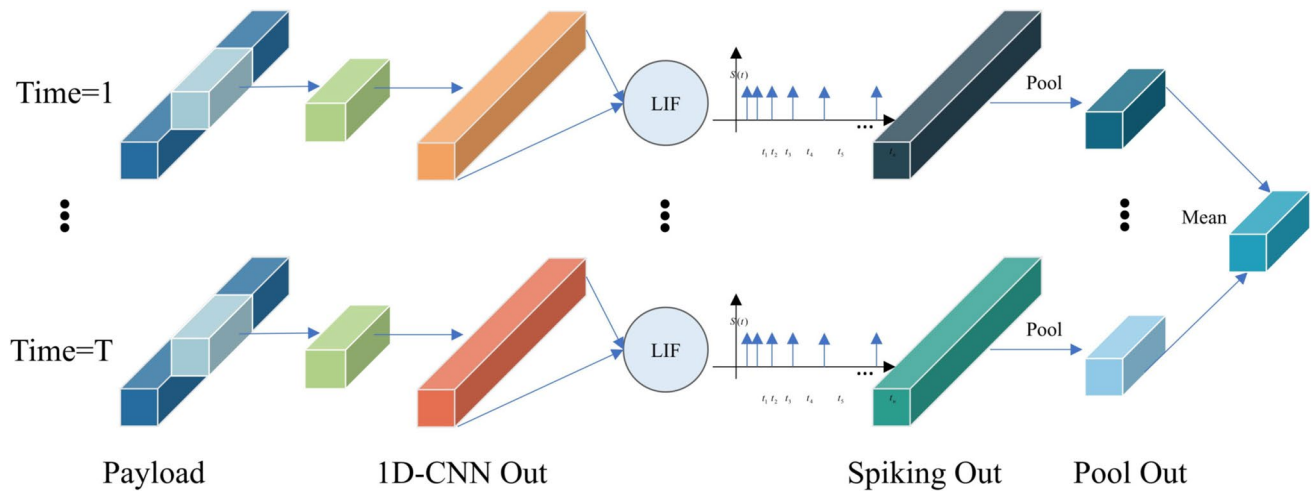


Fig. 2 Spiking convolutional encoder workflow

The spiking convolutional encoder can be expressed by the following equation:

$$\text{encoder}_p(p) = \text{MaxPool}(\text{LIF}(\text{Batchnorm}(\text{Conv1d}(p)))) \quad (5)$$

where $p \in R^{d_2 \times d_1}$ is the embedded payload vector, and $\text{encoder}_p(p)$ is the output of the packet payload vector through the convolutional encoder, LIF is the spiking neuron.

Compared to using CNNs alone, which excel at capturing spatial features, the incorporation of SNNs enhances the ability to mine temporal features. For example, in the case of burst traffic, when encrypted payloads contain consecutive high-density bytes, LIF neurons can rapidly accumulate potentials and trigger spikes. By integrating with CNNs, this approach effectively captures temporal characteristics, whereas using CNNs alone relies on deep convolutions to indirectly capture temporal patterns.

3.4 Cross-gating module based on spiking neurons

Certain fields in the packet header, such as payload protocol and payload length, are directly correlated with payload features. To effectively leverage this interrelated information between the header and payload, we have designed a cross-gating module based on spiking neurons that fuses these features while further reducing the network's energy consumption. The operational mechanism of this module mimics mutual attention, involving average pooling followed by two linear layers interspersed with spiking neurons, and concluding with a SoftMax operation. This transition from pooling to linear layers primarily serves as a filtering mechanism, which removes less relevant information from the message and protocol, preserves key details, and transforms this information into attention weights via the SoftMax

operation. These attention weights are then applied to both feature vectors.

The cross-gating module can be expressed as:

$$\text{attn}_h = \text{Softmax}(w_{h_2}^T (\text{LIF}(w_{h_1}^T (\text{AvgPool}(h_{en})) + b_{h_1})) + b_{h_2}) \quad (6)$$

$$\text{attn}_p = \text{Softmax}(w_{p_2}^T (\text{LIF}(w_{p_1}^T (\text{AvgPool}(p_{en})) + b_{p_1})) + b_{p_2}) \quad (7)$$

$$z = \text{CONCAT}(\text{attn}_h \odot p_{en}, \text{attn}_p \odot h_{en}) \quad (8)$$

Specifically, h_{en} and $p_{en} \in R^{L \times d}$ denote the final encoded outputs of the header vector and payload vector, respectively. attn_h and $\text{attn}_p \in R^{d_2 \times d_1}$ represent the attention weights computed for header and payload features. The symbol $\odot$ indicates element-wise multiplication, and z represents the fused feature output, which is used for subsequent classification tasks.

3.5 Classifier

The classifier is composed of two linear layers. It first flattens the feature vectors into a one-dimensional array, and then the linear layers output the classification results, as expressed by the following equation:

$$f = w_{z_2}^T (w_{z_1}^T (\text{Flatten}(z)) + b_{z_1}) + b_{z_2} \quad (9)$$

Here, f denotes the classification result.

In Spiking Neural Networks, for classification tasks, the Cross Entropy (CE) loss optimizes the model by maximizing the probability of the true label, with its gradient dependent on the derivative of the softmax function. When the SNN output approaches 0 while the true label is 1, the logarithmic term in the CE loss increases sharply, leading to gradient explosion. This nonlinear behavior under extreme

values is exacerbated in SNNs' sparse output scenarios, amplifying training instability. In contrast, the Minimum Square Error (MSE) loss exhibits linear gradient behavior, enabling a more stable training process. Hence, this study adopts MSE as the training loss function, as expressed by the following equation:

$$\mathcal{L} = \mathcal{MSE}(f, y) \quad (10)$$

In this context, y denotes the true labels, and $\mathcal{MSE}$ refers to the Mean Squared Error.

4 Experiments

4.1 Experimental settings

4.1.1 Datasets

To accurately evaluate SCNNTraffic's performance on stream-level classification tasks, we utilized four publicly available datasets for training and testing: ISCX VPN-nonVPN [28] and ISCX Tor-nonTor [29]. ISCX VPN-nonVPN dataset includes both the ISCX VPN and ISCX nonVPN datasets. The former contains traffic collected through VPN, which is difficult to identify due to obfuscation techniques, while the latter consists of regular traffic that is not collected via VPN. Similarly, ISCX Tor-nonTor dataset includes the ISCX Tor and ISCX nonTor datasets. The ISCX Tor dataset is collected through Tor, making its traffic hard to trace. Additionally, ISCX nonTor also represents regular traffic that is not collected over the Tor network.

During data processing, we initially employ the SplitCap tool to partition network flows and filter out ineffective or erroneous flows. The number of preprocessed samples for each dataset is detailed in Appendix A. It should be noted that because the ISCX Tor dataset yields few flows when divided by five-tuple, we opt to re-partition the data every 60 seconds [30]. Next, we remove user-sensitive information from the headers, such as MAC and IP addresses, and delete service packets lacking useful information. Each network flow retains only the first 200 bytes, if fewer than 200 bytes are present, we pad with the value 256 as a mask until

the byte count meets the requirement, thereby distinguishing it from the standard byte value range of 0-255.

4.1.2 Experimental settings

To enhance the firing rate of spiking neurons, we typically set an observation time T to observe the input multiple times. This increased spike activity raises the probability of accurately identifying specific categories. Due to the non-differentiable nature of the step function impedes training of SNNs, we employ a surrogate gradient function to substitute for the step function, thereby enabling effective training of spiking neurons. In this study, we set the observation time T to 2 and choose the Tanh function as the surrogate gradient function. Comparative experiments with varying observation times and different surrogate functions are conducted to analyze their effects. Considering the inherent characteristics of the model based on SNN and its lightweight architecture design, it already possesses a strong resistance to overfitting. Therefore, traditional regularization strategies such as Dropout and weight decay were not introduced, so as to avoid disrupting the expression of spike features and maintain inference efficiency.

During the training phase, we set the maximum number of epochs to 100. We use the Adam optimizer with a learning rate scheduler that gradually decays the learning rate from $1e-2$ to $1e-5$. The batch size is 16, the observation time T is 2, the surrogate gradient function is the Tanh function, and the reset method is soft reset. We implemented all models using PyTorch and ran each experiment independently 10 times on a single Nvidia Tesla V100 GPU to obtain mean values and standard deviations. To ensure fair comparisons, we utilized four metrics: overall accuracy (AC), precision (PR), recall (RC), and macro F1 score (F1). Our model was evaluated against the latest baselines, including machine learning-based methods (e.g., AppScanner [4], BIND [10], CUMUL [31]) and deep learning-based methods (e.g., DF [32], GraphDApp [33], ET-BERT [34], CLE-TFE [5]).

4.2 Comparative experiments

Tables 1 and 2 present the comparative results on the public datasets ISCX VPN-nonVPN and ISCX Tor-nonTor.

Table 1 Classification accuracy (%) comparison among seven ETC methods on public ISCX VPN-nonVPN and ISCX Tor-nonTor datasets. Results format: Avg.(±Std.). Boldface entries indicate the best performance, and underlined entries indicate the second-best performance among all compared methods.

Model	ISCX VPN	ISCX nonVPN	ISCX Tor	ISCX nonTor
AppScanner	86.93(±0.04)	74.52(±0.04)	89.13(±0.02)	92.14(±0.02)
BIND	92.21(±0.02)	74.90(±0.03)	90.44(±0.02)	95.70(±0.02)
CUMUL	68.32(±0.02)	58.07(±0.02)	70.31(±0.02)	88.84(±0.02)
DF	84.05(±0.52)	65.87(±0.43)	81.23(±0.26)	94.72(±0.22)
GraphDApp	75.39(±0.65)	68.91(±0.58)	78.91(±0.38)	92.83(±0.27)
ET-BERT	<u>97.42(±0.11)</u>	90.66(±0.09)	95.81(±0.04)	96.46(±0.04)
CLE-TFE	97.28(±0.04)	<u>92.21(±0.05)</u>	99.98(±0.02)	<u>97.14(±0.02)</u>
SCNNTraffic	98.65(±0.18)	92.51(±0.22)	<u>99.24(±0.08)</u>	98.38(±0.04)

Table 2 Comparison of different methods in precision (%), Recall (%) and F1-score (%). Boldface entries indicate the best performance, and underlined entries indicate the second-best performance among all compared methods.

Model	ISCX VPN				ISCX nonVPN				ISCX Tor				ISCX nonTor			
	PR	RC	F1		PR	RC	F1		PR	RC	F1		PR	RC	F1	
AppScanner	87.26	86.93	86.86		74.04	74.52	73.86		90.67	89.13	88.97		91.46	92.14	91.23	
BIND	92.20	92.21	92.15		74.87	74.90	74.51		92.02	90.44	89.68		95.32	95.70	95.41	
CUMUL	69.79	68.32	68.72		58.92	58.07	58.17		73.26	70.31	71.21		90.79	88.84	89.72	
DF	85.48	84.05	84.41		72.60	65.87	65.77		80.77	81.23	80.47		94.32	94.72	94.48	
GraphDApp	75.99	75.39	74.54		67.92	68.91	68.08		80.05	78.91	79.23		92.99	92.83	92.87	
ET-BERT	97.51	97.42	97.43		90.79	90.66	91.64		94.09	95.81	95.55		96.25	96.46	96.04	
CLE-TFE	97.39	97.28	97.21		92.67	92.21	92.18		99.97	99.98	99.98		<u>97.93</u>	<u>97.14</u>	<u>97.08</u>	
SCNNTraffic	98.67	98.65	98.65		<u>92.59</u>	92.51	92.52		<u>99.27</u>	<u>99.24</u>	<u>99.24</u>		98.24	98.38	98.27	

Based on these results, we can derive the following observations: (1) On the ISCX VPN and ISCX nonTor datasets, SCNNTraffic outperforms the baseline methods across all four metrics, achieving accuracy improvements of 1.23% and 1.24%, respectively, over the previous best models. On the ISCX nonVPN dataset, our method achieves slightly better performance in three of the four metrics by 0.30%, 0.30%, and 0.34%, respectively, except for precision which is 0.08% lower than the CLE-TFE model. This advantage stems from our framework that separately processes packet headers and payloads, along with the robust handling of time-varying sequences by spiking neurons. On the ISCX Tor dataset, our method's performance is closely comparable to CLE-TFE, with respective differences of -0.74%, -0.70%, -0.74%, and -0.74%. This is likely because Tor traffic employs multi-layer encryption and onion routing technologies, which reduce the effectiveness of the header structural features that SCNNTraffic relies on. In contrast, CLE-TFE's packet-level graph contrastive learning task enhances inter-packet relationships, thereby achieving superior performance on this dataset. (2) The superior performance of ET-BERT can be attributed to its extensive parameter scale and pre-training on large datasets. However, this comes at the cost of significantly larger parameter counts and computational demands. CLE-TFE also employs separate inputs for headers and payloads, leveraging the strong representation power of Graph Neural Networks(GNNs) and graph contrastive learning to achieve high accuracy. Similar to ET-BERT, it has substantially larger parameter counts and higher computational requirements compared to our proposed method. SCNNTraffic excels due to the robust time-series processing capabilities of SCNNs. By efficiently capturing temporal dynamics and reducing computational overhead, our approach offers a compelling balance between performance and efficiency, making it particularly suitable for edge deployment and resource-constrained environments. (3) The standard deviation results in Table 1 show that the standard deviation of SCNNTraffic ranges from 0.04% to 0.22%, which is within an acceptable range compared to the performance gains. This demonstrates that our method is indeed superior to the comparative baselines, rather than being a result of random fluctuations.

4.3 Analysis of computational cost and model parameters

We evaluated the computational cost of the proposed model on the ISCX VPN dataset (with a batch size of 16) by computing metrics such as the number of floating-point operations (FLOPs), energy consumption, number of parameters, and average runtime per iteration using the SyOPs tool [35].

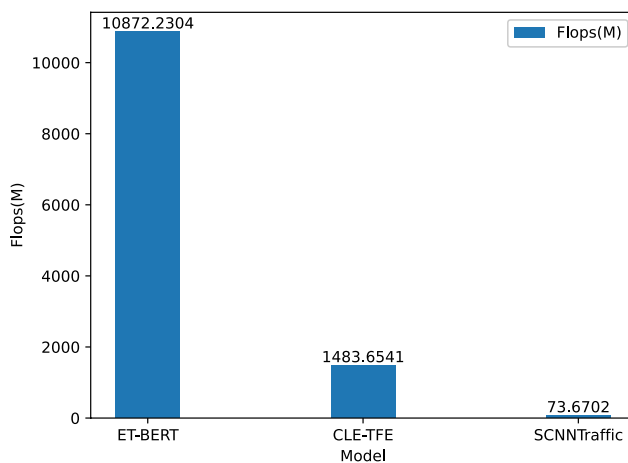
We compare these results with those of ET-BERT and CLE-TFE, as illustrated in Fig. 3.

From the results in Fig. 3, we can derive the following conclusions: (1) ET-BERT, as a pre-trained model, incurs the highest computational cost among the three models evaluated. Its large architecture and complex operations result in significantly higher FLOPs and parameter counts, making it less efficient for resource-constrained environments. (2) Our proposed model achieves the lowest computational cost, with FLOPs and parameter counts being merely 4.97% and 2.09% of those of the CLE-TFE model, respectively. This is notably lower than both ET-BERT and CLE-TFE, making our model more suitable for practical edge deployment where computational resources are limited. (3) SCNNTraffic consumes only 0.3248 mJ of energy per inference, significantly less than the two comparative models. This superior energy efficiency is attributed to the

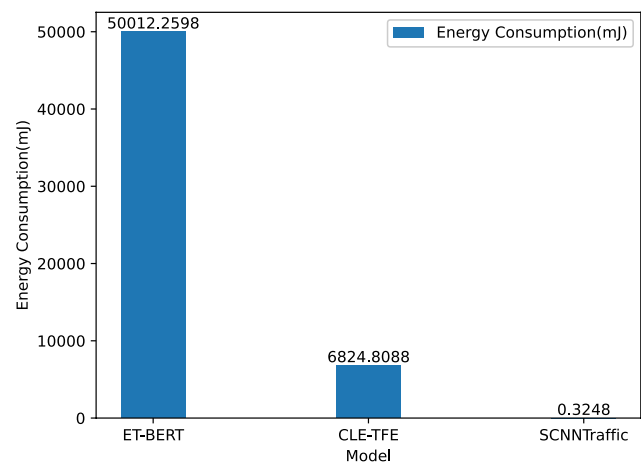
inherent properties of spiking neurons, which operate with sparse activations and event-driven mechanisms, thereby reducing power consumption.

4.4 Ablation study

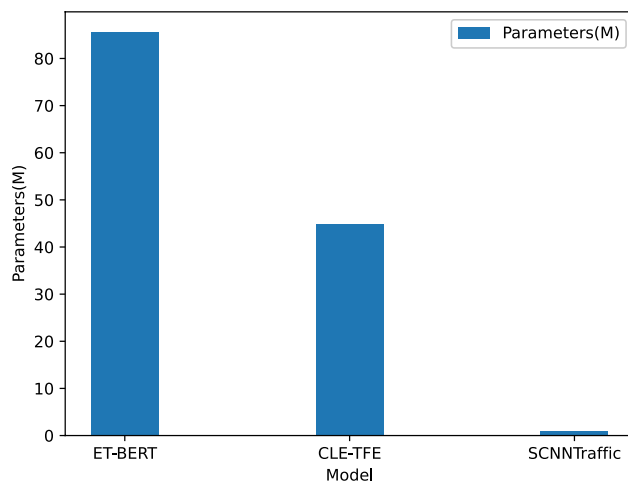
In this section, we perform ablation studies on the ISCX VPN and ISCX Tor datasets, with the results presented in Table 3. For simplicity, we denote the packet header input, payload input, spiking neurons for headers, spiking neurons for payloads, and the cross-gating fusion module as "P", "H", "PLIF", "HLIF", and "CGF", respectively. Furthermore, we not only validate the effectiveness of these modules but also examine the impacts of various alternative modules or operations, such as average pooling ("AP"), adaptive average pooling ("AAP") and adaptive maximum pooling ("AMP") in spike convolutional encoding, convolutional stride are 3



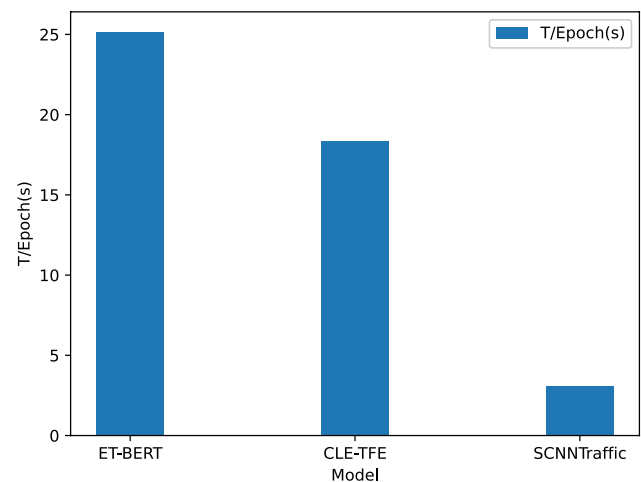
(a) FLOPs.



(b) Energy Consumption.



(c) Parameters.



(d) Training Time per Epoch.

Fig. 3 Analysis of computational cost results

Table 3 Results of ablation study on public ISCX VPN and Tor datasets. Boldface entries indicate the best performance, and underlined entries indicate the second-best performance among all compared methods

Method	P	H	PLIF	HLIF	CG	AC	PR	RC	F1
w/o P	X	✓	✓	X	✓	95.95 98.18	96.06 98.25	95.95 98.18	95.88 98.16
w/o H	✓	X	✓	X	✓	83.39 94.27	83.55 94.42	83.39 94.27	83.10 94.14
w/o PLIF	✓	✓	X	X	✓	97.99 98.52	98.02 98.57	97.99 98.52	97.99 98.51
w/ HLIF	✓	✓	✓	✓	✓	97.47 <u>99.21</u>	97.55 <u>99.25</u>	97.47 <u>99.21</u>	97.48 <u>99.21</u>
w/o CG	✓	✓	✓	X	X	97.60 98.80	97.64 98.83	97.60 98.80	97.59 98.80
w/ AP	✓	✓	✓	X	✓	93.52 97.97	94.04 98.04	93.52 97.97	93.59 97.97
w/ AAP	✓	✓	✓	X	✓	93.94 97.88	94.50 97.91	93.94 97.88	94.02 97.87
w/ AMP	✓	✓	✓	X	✓	97.69 97.56	97.71 97.65	97.69 97.56	97.68 97.54
w/ CE	✓	✓	✓	X	✓	97.60 98.80	97.63 98.05	97.60 98.80	97.60 98.80
w/ EMA	✓	✓	✓	X	✓	<u>98.03</u> 99.14	<u>98.08</u> 99.17	<u>98.03</u> 99.14	<u>98.03</u> 99.14
w/ PS	✓	✓	✓	X	✓	97.72 98.70	97.76 98.73	97.72 98.70	97.72 98.70
SCNNTraffic	✓	✓	✓	X	✓	98.65 99.24	98.67 99.27	98.65 99.24	98.65 99.24

and 7 ("S=3/7"), using cross-entropy loss ("CE"), adding a multi-scale attention module ("EMA"), and encoder parameter sharing("PS").

Based on the results in Table 3, we can draw the following conclusions: (1) Removing the payload input reduces accuracy by 2.7% and 1.06% on the ISCX VPN and Tor datasets, respectively, while removing the header input reduces accuracy by 15.26% and 4.97%. These findings indicate that packet headers play a more significant role in classification than payloads, especially on the ISCX Tor dataset where the impact is markedly higher. (2) Removing spiking neurons from the spike convolutional encoder for payloads and adding them to the convolutional encoder for headers reduces accuracy by 0.66% and 1.18% on the ISCX VPN dataset, and by 0.72% and 0.03% on the ISCX Tor dataset. This suggests that spiking neurons are particularly effective for handling highly variable data like payloads but offer limited benefits when applied to less variable data such as headers. (3) Adding the cross-gating fusion module increases accuracy by 1.05% and 0.44% on the ISCX VPN and Tor datasets, respectively, demonstrating its general effectiveness in enhancing classification performance.

The remaining experiments in Table 3, which involve alternative pooling methods, loss functions, and the addition of the EMA module, lead to the following conclusions: (1) Both average pooling and two adaptive pooling reduce the accuracy of the two datasets, and the maximum pooling achieves the best effect. (2) Using cross-entropy loss decreases accuracy by 1.05% and 0.44% on the two datasets, indicating that MSE is more stable as a loss function for classification tasks in SNNs. (3) Adding the multi-scale attention module reduces classification accuracy by 0.62% and 0.1% on the ISCX VPN and Tor datasets, respectively. This indicates that our spiking encoder can adequately

learn data features without requiring an additional attention module. The inherent mechanisms within the spiking convolutional network are sufficient for feature extraction, reducing the need for complex attention layers. (4) After sharing encoder parameters, the classification accuracy decreased by 0.93% and 0.54%, respectively. Although sharing encoder parameters can further reduce the number of model parameters, the forced parameter sharing impairs the effectiveness of feature extraction due to the disparity in features between header data and payload data.

Additionally, we have provided comparative results of different pulse encoding methods, which can be found in Appendix B.

4.5 Comparison of different observation times, convolution stride, surrogate gradient functions, and reset methods

The observation time T is a crucial parameter in SNNs. Shorter observation times can significantly reduce computational complexity but may inadequately capture the full dynamics of neuronal changes, potentially leading to less accurate representations and classification. Conversely, longer observation times provide a more comprehensive view of neuronal activity, which can enhance precision but at the cost of increased computational demands. Therefore, selecting an appropriate observation time T is essential to strike a balance between enhancing precision and minimizing computational resource usage. Optimizing T ensures that the network can effectively process temporal information while remaining efficient, making it a key consideration in the design and application of SNNs.

In this experiment, we set the surrogate gradient function to the Tanh function and the reset method to soft reset, and

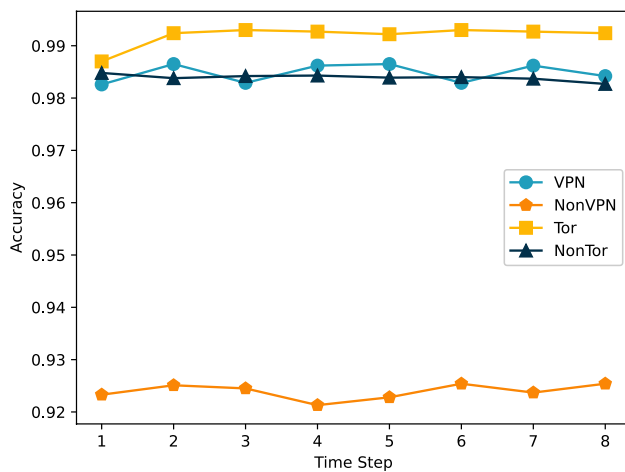


Fig. 4 Evaluation metrics of SCNNTraffic on four datasets under different observation times T

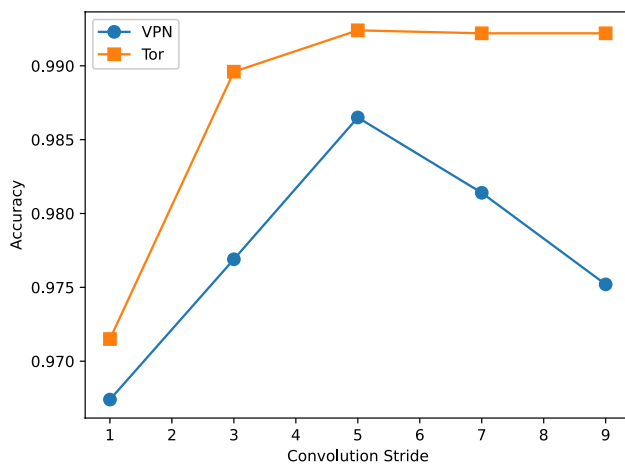


Fig. 5 Accuracy of SCNNTraffic on two datasets under different convolution stride

examine the four evaluation metrics on the ISCX datasets for observation times ranging from 1 to 8. The results are illustrated in Fig. 4.

Based on the results shown in Fig. 4, the evaluation metrics for our model on the ISCX VPN dataset fluctuate with observation time, peaking at T=2 and T=7; on the ISCX NonVPN dataset, the metrics decrease as observation time

increases, being highest at T=1; on the ISCX Tor dataset, the metrics initially increase and then stabilize, peaking at T=2; on the ISCX NonTor dataset, the metrics remain relatively stable. In conclusion, an observation time T=2 enables the model to achieve higher classification accuracy while using fewer computational resources.

The convolutional stride directly affects the downsampling granularity of traffic bytes. We systematically analyzed the model's performance on VPN and Tor datasets under various convolutional stride conditions, with experimental results shown in Fig. 5. The analysis reveals that the model achieves optimal performance when the convolutional stride is set to 5, while excessively large or small strides both lead to performance degradation.

In this section, we also analyze the impact of different surrogate gradient functions and reset methods for spiking neurons, as presented in Table 4. Based on the findings in Table 4, we can derive the following conclusions: (1) Soft reset consistently outperforms hard reset. (2) The Tanh function is the optimal choice for a surrogate gradient function, followed closely by the Sigmoid function. The Softsign function yields the poorest performance.

4.6 Visualization analysis of packet encoding and payload spiking encoding

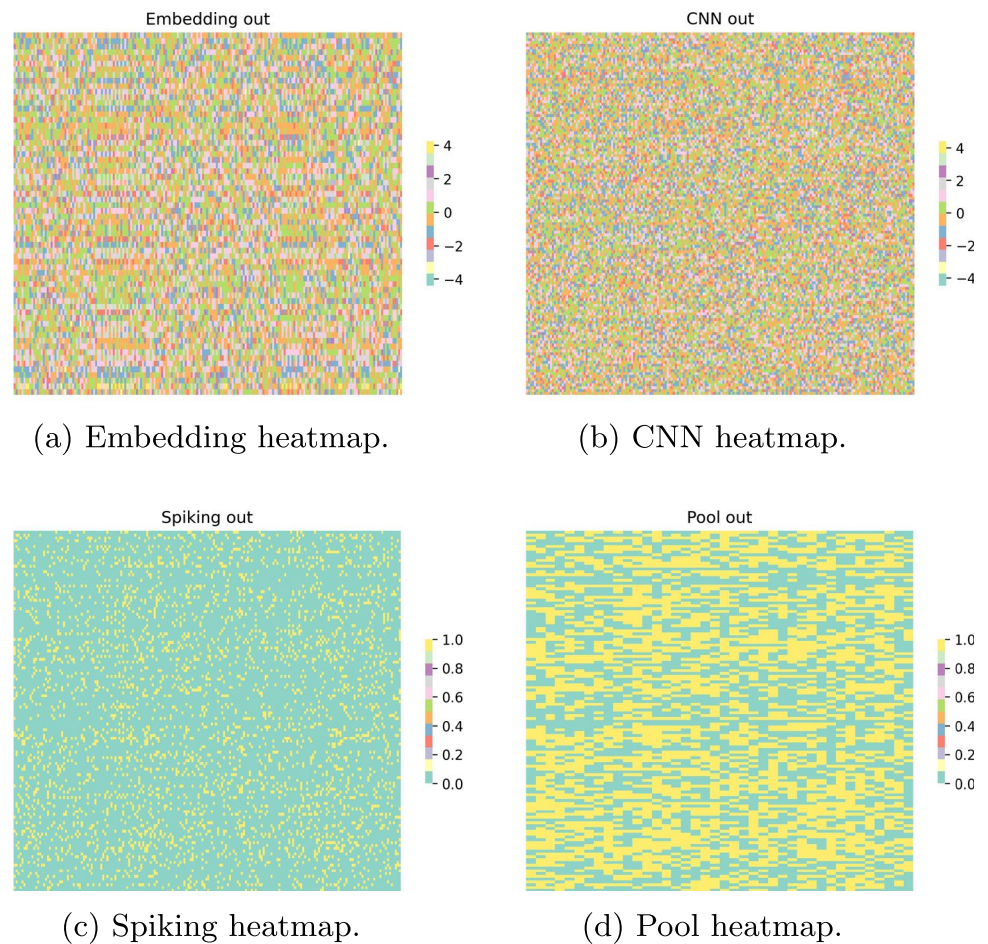
In this study, we analyze the operation of the spiking encoder by presenting heatmaps at various stages: after word embedding, convolution, spiking, and pooling, as illustrated in Fig. 6. These sequential visualizations provide insights into how features are processed and transformed through each layer of the spiking encoder.

From Fig. 6a and b, we observe that: the spiking encoder first learns initial features through the convolution layer. This stage captures local patterns from the input data, forming the basis for subsequent processing. Following convolution, these extracted features are converted into spikes using spiking neurons (Fig. 6c). Notably, the heatmap from the spiking neuron outputs shows that only a few spikes are initially activated, indicating that most spiking neurons remain silent at this early stage. This sparse activity suggests that

Table 4 Surrogate functions and reset methods comparison. Boldface entries indicate the best performance, and underlined entries indicate the second-best performance among all compared methods

Method	Reset method	Surrogate functions	AC	PR	RC	F1
1	Hard	Sigmoid	98.32 99.19	98.35 99.21	98.32 99.19	98.32 99.19
2	Hard	Softsign	98.19 99.14	98.23 99.17	98.23 99.14	98.23 99.14
3	Hard	Tanh	<u>98.49</u> 99.19	98.51 99.21	98.49 99.19	<u>98.48</u> 99.19
4	Soft	Sigmoid	<u>98.49</u> <u>99.22</u>	<u>98.54</u> <u>99.24</u>	<u>98.49</u> <u>99.22</u>	<u>98.48</u> <u>99.22</u>
5	Soft	Softsign	98.42 99.19	98.48 99.21	98.42 99.19	<u>98.48</u> 99.19
6(default)	Soft	Tanh	98.65 99.24	98.67 99.27	98.65 99.24	98.65 99.24

Fig. 6 Heatmap displaying the results of the spiking encoder



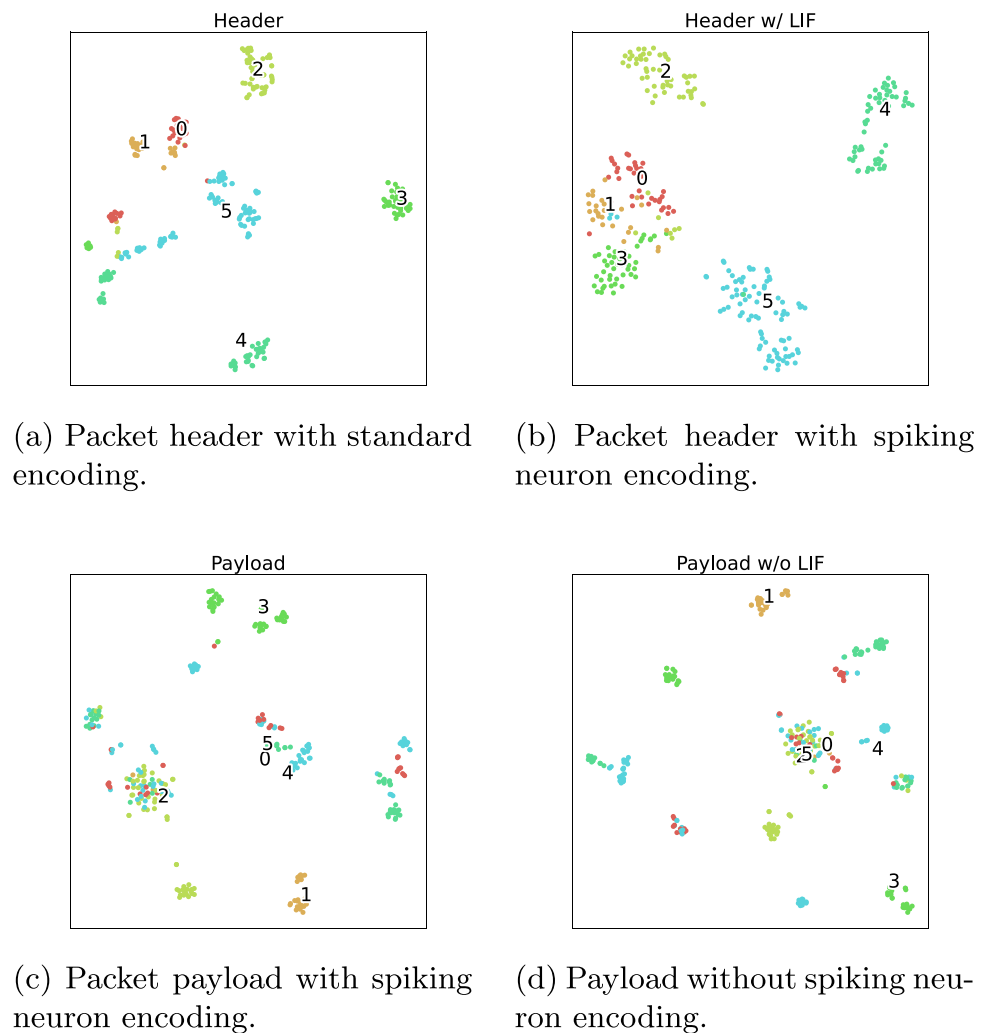
the network is selectively responding to specific features, which can be beneficial for reducing noise and focusing on relevant information. Finally, the pooling layer both down-samples the data and increases the density of spike activations. As shown in the last image of Fig. 6d, after passing through the pooling layer, the yellow regions representing the spikes become predominantly visible. This indicates that pooling not only reduces the spatial dimensions but also enhances the representation of significant features by increasing spike activity.

To better illustrate the effectiveness of spiking neurons and the distinct impacts of headers and payloads, we perform t-SNE visualization on the encoded output of the ISCX VPN test dataset, as shown in Fig. 7. The t-SNE visualization provides a two-dimensional representation that helps in understanding the high-dimensional data structure generated by the spiking encoder.

From the results in Fig. 7, we can derive the following conclusions: (1)Header encoding leads to greater distances between different classes in the embedding space compared to payload encoding, resulting in clearer decision boundaries. This indicates that packet headers play a

more crucial role in classification. The structured nature of headers allows SCNNTraffic to effectively distinguish between various types of network traffic, making them particularly valuable for identifying distinct classes. (2)Incorporating spiking neurons into header encoding increases intra-class distances in the embedding space, which paradoxically blurs the decision boundaries between different classes. This indicates that, although spiking neurons introduce variability within each class, they might diminish the model's overall discriminative power when applied exclusively to headers. It implies that spiking neurons might add unnecessary complexity to an already well-separated feature space, potentially degrading performance. (3)Incorporating spiking neurons into payload encoding increases inter-class distances in the embedding space, thereby enhancing the robustness of the model. Since payloads contain more variable and complex data, spiking neurons can help capture subtle patterns that are not easily discernible through traditional methods. This enhancement improves the model's ability to differentiate between classes, especially when dealing with the nuanced information contained in payloads.

Fig. 7 t-SNE visualization of ISCX VPN test dataset



5 Conclusion

This study introduces SCNNTraffic, an encrypted traffic classification method based on SCNNs. By leveraging spiking convolutional encoding and feature fusion modules, the method effectively extracts packet features, achieving robust classification performance with a lightweight and low-power design. The experiments evaluated the proposed method against mainstream baseline models. On four datasets, our method generally surpasses or closely matches models with larger parameter sizes while substantially reducing computational costs and energy consumption. This validates SCNNTraffic's effectiveness and lightweight characteristics, making it valuable for edge deployment in network data analysis.

The performance of SCNNTraffic is challenged by modern encryption techniques that use random byte padding

to obfuscate traffic fingerprints, thereby compromising the model's classification accuracy. Furthermore, the model's practical deployment is constrained by its heavy reliance on substantial volumes of labeled data for training. Looking ahead, we aim to improve the model through the following methods: (1) Addressing the loss of packet interaction direction information in the current text-based data representation is crucial. Incorporating this temporal and directional context can enhance the model's ability to understand the flow of network traffic, potentially leading to optimal classification performance. (2) We plan to integrate spiking neurons with pre-trained models to leverage transfer learning benefits. This approach aims to enhance classification performance under conditions with limited labeled samples, thereby broadening the applicability of the SNN-based method in scenarios where obtaining large annotated datasets is challenging.

Appendix A: Dataset statistics

This section provides statistical details about the datasets used in this study, including class names and class numbers.

Table 5 ISCX VPN dataset statistics

Class names	Class numbers
Chat	198
Email	143
File	320
P2P	235
Streaming	402
VoIP	467

Table 6 ISCX NonVPN dataset statistics

Class names	Class numbers
Chat	338
Email	232
File	1111
Streaming	811
Video	497
VoIP	905

Table 7 ISCX Tor dataset statistics

Class names	Class numbers
Audio	329
Browsing	383
Chat	74
File	159
Mail	54
P2P	374
Video	187
VoIP	399

Table 8 ISCX NonTor dataset statistics

Class names	Class numbers
Audio	1000
Browsing	16052
Chat	189
Email	147
FTP	1577
P2P	15658
Video	874
VoIP	217

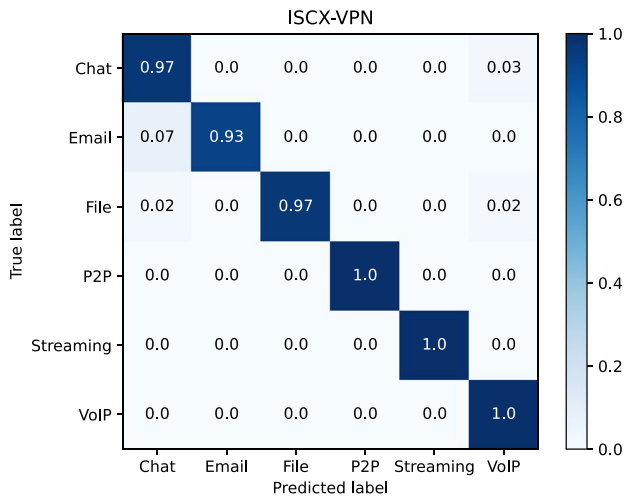
Appendix B: Encoding comparison

In this section, we compared the impact of different spiking encoding methods—rate coding and temporal coding—on the experimental results. Specifically, for rate coding, we employed the commonly used Poisson Encoding (PE) in SpikingJelly, while for temporal coding, we utilized Latency Encoding (LT) with both linear and logarithmic strategies. The comparative results, as shown in Table 9, indicate that the accuracy rates decreased by 3.84% and 13.49% on the two datasets when using rate coding and temporal coding, respectively. This demonstrates that the embedding method adopted in this paper is more suitable for representing encrypted traffic data.

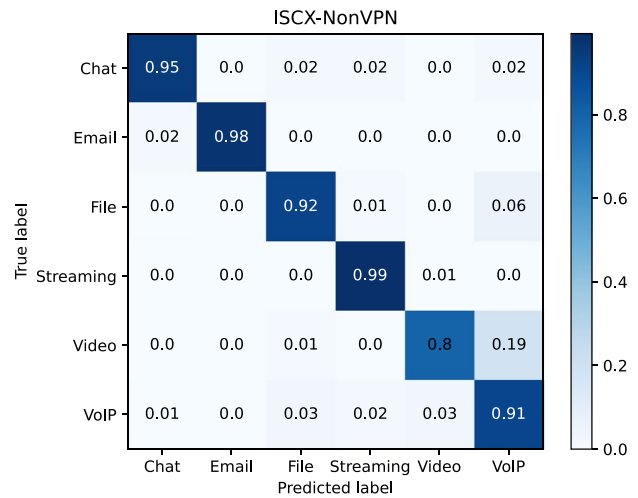
Table 9 Performance comparison of different encoding methods on the ISCX VPN and ISCX Tor datasets. Boldface entries indicate the best performance

Model	ISCX VPN				ISCX Tor			
	AC	PR	RC	F1	AC	PR	RC	F1
PE	94.81	94.78	94.81	94.77	79.68	79.31	79.68	79.52
LE (Linear)	90.26	90.03	90.26	90.09	85.75	85.60	85.75	85.26
LE (Log)	90.91	91.38	90.91	90.78	82.64	82.64	82.64	82.17
SCNNTraffic	98.65	98.67	98.65	98.65	99.24	99.27	99.24	99.24

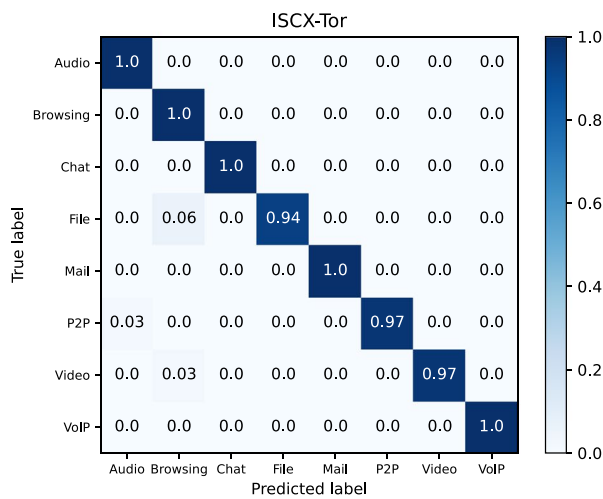
Appendix C: Confusion matrix results



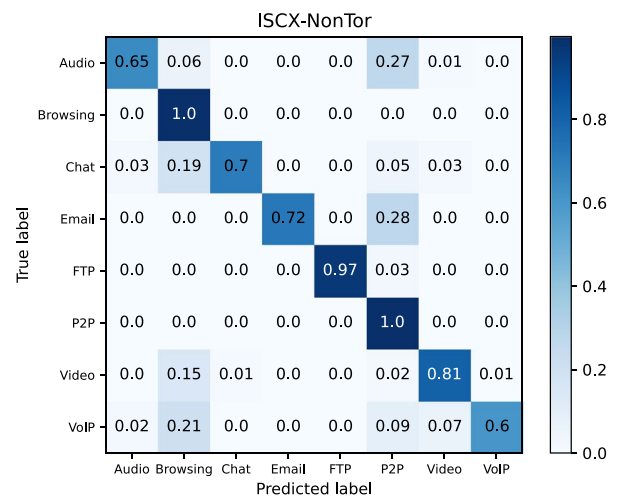
(a) Confusion matrix of ISCX VPN.



(b) Confusion matrix of ISCX nonVPN.



(c) Confusion matrix of ISCX Tor.



(d) Confusion matrix of ISCX nonTor.

Fig. 8 Confusion matrix of four datasets

Author Contributions Q.Z and D.Q came up this method and wrote the main manuscript text, H.Z and Y.C verified the method, W.Z polished the manuscript.

Funding This work was supported by the National Natural Science Foundation of China (No.62171470), Natural Science Foundation of Henan Province (No.232300421240), and Henan Zhongyuan Science and Technology Innovation Leading Talent Project (No.234200510019).

Data Availability No datasets were generated or analysed during the current study.

Materials Availability Not applicable.

Code availability Not applicable.

Declarations

Competing Interests The authors declare no competing interests.

Ethics Approval and Consent to Participate Not applicable.

Consent for Publication Not applicable.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Yang B, Liu D (2019) Research on network traffic identification based on machine learning and deep packet inspection. In: 2019 IEEE 3rd Information technology, networking, electronic and automation control conference (ITNEC), pp 1887–1891. <https://doi.org/10.1109/ITNEC.2019.8729153>
2. Bujlow T, Carela-Español V, Barlet-Ros P (2015) Independent comparison of popular DPI tools for traffic classification. *Comput Netw* 76:75–89. <https://doi.org/10.1016/J.COMNET.2014.11.001>
3. Singh Popli M, Singh RP, Kaur Popli N, Mamun M (2025) A federated learning framework for enhanced data security and cyber intrusion detection in distributed network of underwater drones. *IEEE Access* 13:12634–12646. <https://doi.org/10.1109/ACCESS.2025.3530499>
4. Taylor VF, Spolaor R, Conti M, Martinovic I (2016) Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic. In: IEEE European symposium on security and privacy, EuroS&P 2016, Saarbrücken, Germany, March 21–24, 2016, pp 439–454. <https://doi.org/10.1109/EUROSP.2016.40>
5. Zhang H, Xiao X, Yu L, Li Q, Ling Z, Zhang Y (2024) One train for two tasks: An encrypted traffic classification framework using supervised contrastive learning. <https://doi.org/10.48550/ARXIV.2402.07501>
6. Luan N, Xiong K, Zhang Y, He R, Qu G, Ai B (2022) 6g: typical applications, key technologies and challenges. *Chin J Int Things* 6(1):29. <https://doi.org/10.11959/j.issn.2096-3750.2022.00253>
7. Akbar MS, Hussain Z, Ikram M, Sheng QZ, Mukhopadhyay SC (2025) On challenges of sixth-generation (6g) wireless networks: A comprehensive survey of requirements, applications, and security issues. *J Netw Comput Appl* 233:104040. <https://doi.org/10.1016/j.jnca.2024.104040>
8. Cheng J, Wu Y, E Y, You J, Li T, Li H, Ge J (2021) Matec: A lightweight neural network for online encrypted traffic classification. *Comput Netw* 199(C). <https://doi.org/10.1016/j.comnet.2021.108472>
9. Xu Y, Cao J, Song K, Xiang Q, Cheng G (2023) Fasttraffic: A lightweight method for encrypted traffic fast classification. *Comput Netw* 235(C). <https://doi.org/10.1016/j.comnet.2023.109965>
10. Al-Naami K, Chandra S, Mustafa A, Khan L, Lin Z, Hamlen KW, Thuraisingham B (2016) Adaptive encrypted traffic fingerprinting with bi-directional dependence. In: Schwab S, Robertson WK, Balzarotti D (eds) Proceedings of the 32nd annual conference on computer security applications, ACSAC 2016, Los Angeles, CA, USA, December 5–9, 2016, pp 177–188
11. Nowroozi E, Haider I, Taheri R, Conti M (2025) Federated learning under attack: Exposing vulnerabilities through data poisoning attacks in computer networks. *IEEE Trans Netw Serv Manage* 22(1):822–831. <https://doi.org/10.1109/TNSM.2025.3525554>
12. Nabavirazavi S, Taheri R, Shojafar M, Iyengar SS (2023) Impact of aggregation function randomization against model poisoning in federated learning. In: 2023 IEEE 22nd International conference on trust, security and privacy in computing and communications (TrustCom), pp 165–172. <https://doi.org/10.1109/TrustCom.2023.00043>
13. Taheri R, Arabikhan F, Gegov A, Akbari N (2024) Robust aggregation function in federated learning. *Advances in Information Systems. Artificial intelligence and knowledge management*. Springer, Cham, pp 168–175
14. Rezaei H, Taheri R, Jordanov I, Shiaeles S (2025) Adapt-lfa: Adaptive gradient-guided label flipping attack against federated learning-based intrusion detection in iot. In: 2025 IEEE international conference on cyber security and resilience (CSR), pp 407–412. <https://doi.org/10.1109/CSR64739.2025.11130071>
15. Wang W, Zhu M, Zeng X, Ye X, Sheng Y (2017) Malware traffic classification using convolutional neural network for representation learning. In: 2017 International conference on information networking, ICOIN 2017, Da Nang, Vietnam, January 11–13, 2017, pp 712–717. <https://doi.org/10.1109/ICOIN.2017.7899588>
16. Zhao R, Zhan M, Deng X, Wang Y, Wang Y, Gui G, Xue Z (2023) Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation. In: Williams B, Chen Y, Neville J (eds) Thirty-Seventh AAAI conference on artificial intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7–14, 2023, pp 5420–5427. <https://doi.org/10.1609/AAAI.V37I4.25674>
17. Zhang H, Yu L, Xiao X, Li Q, Mercaldo F, Luo X, Liu Q (2023) TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. <https://doi.org/10.48550/ARXIV.2307.16713>
18. Han X, Xu G, Zhang M, Yang Z, Yu Z, Huang W, Meng C (2024) DE-GNN: dual embedding with graph neural network

- for fine-grained encrypted traffic classification. *Comput Netw* 245:110372. <https://doi.org/10.1016/J.COMNET.2024.110372>
19. Roy K, Jaiswal A, Panda P (2019) Towards spike-based machine intelligence with neuromorphic computing. *Nature* 575(7784):607–617. <https://doi.org/10.1038/s41586-019-1677-2>
 20. Neftci EO, Mostafa H, Zenke F (2019) Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Process Mag* 36(6):51–63. <https://doi.org/10.1109/MSP.2019.2931595>
 21. Barchid S, Mennesson J, Eshraghian J, Djeraba C, Bennamoun M (2023) Spiking neural networks for frame-based and event-based single object localization. *Neurocomputing* 559:126805. <https://doi.org/10.1016/J.NEUCOM.2023.126805>
 22. Kim SJ, Park S, Na B, Yoon S (2020) Spiking-yolo: Spiking neural network for energy-efficient object detection. In: The thirty-fourth AAAI conference on artificial intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7–12, 2020, pp 11270–11277. <https://doi.org/10.1609/AAAI.V34I07.6787>
 23. Yang Q, Liu Q, Li N, Ge M, Song Z, Li H (2024) SVAD: A robust, low-power, and light-weight voice activity detection with spiking neural networks. In: IEEE international conference on acoustics, speech and signal processing, ICASSP 2024, Seoul, Republic of Korea, April 14–19, 2024, pp 221–225. <https://doi.org/10.1109/ICASSP48485.2024.10446945>
 24. Song Z, Wu J, Zhang M, Shou MZ, Li H (2024) Spiking-leaf: A learnable auditory front-end for spiking neural networks. In: IEEE international conference on acoustics, speech and signal processing, ICASSP 2024, Seoul, Republic of Korea, April 14–19, 2024, pp 226–230. <https://doi.org/10.1109/ICASSP48485.2024.10446789>
 25. Zhang Y, Xiang S, Jiang S, Han Y, Guo X, Zheng L, Shi Y-C, Hao Y (2023) Hybrid photonic deep convolutional residual spiking neural networks for text classification. *Opt Express* 31(17):28489–28502
 26. Zhu R, Zhao Q, Li G, Eshraghian J (2024) Spikeqpt: Generative pre-trained language model with spiking neural networks. *Trans Mach Learn Res* 2024
 27. Izhikevich EM (2004) Which model to use for cortical spiking neurons? *IEEE Trans Neural Netw* 15(5):1063–1070. <https://doi.org/10.1109/TNN.2004.832719>
 28. Draper-Gil G, Lashkari AH, Mamun MSI, Ghorbani AA (2016) Characterization of encrypted and VPN traffic using time-related features. In: Camp O, Furnell S, Mori P (eds) Proceedings of the 2nd international conference on information systems security and privacy, ICISSP 2016, Rome, Italy, February 19–21, 2016, pp 407–414. <https://doi.org/10.5220/0005740704070414>
 29. Lashkari AH, Draper-Gil G, Mamun MSI, Ghorbani A.A (2017) Characterization of tor traffic using time based features. In: Mori P, Furnell S, Camp O (eds) Proceedings of the 3rd international conference on information systems security and privacy, ICISSP 2017, Porto, Portugal, February 19–21, 2017, pp 253–262. <https://doi.org/10.5220/0006105602530262>
 30. Shapira T, Shavitt Y (2021) Flowpic: A generic representation for encrypted traffic classification and applications identification. *IEEE Trans Netw Serv Manag* 18(2):1218–1232. <https://doi.org/10.1109/TNSM.2021.3071441>
 31. Panchenko A, Lanze F, Pennekamp J, Engel T, Zinnen A, Henze M, Wehrle K (2016) Website fingerprinting at internet scale. In: 23rd Annual network and distributed system security symposium, NDSS 2016, San Diego, California, USA, February 21–24, 2016
 32. Sirinam P, Imani M, Juarez M, Wright M (2018) Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In: Lie D, Mannan M, Backes M, Wang X (eds) Proceedings of the 2018 ACM SIGSAC conference on computer and communications security, CCS 2018, Toronto, ON, Canada, October 15–19, 2018, pp 1928–1943. <https://doi.org/10.1145/3243768>
 33. Shen M, Zhang J, Zhu L, Xu K, Du X (2021) Accurate decentralized application identification via encrypted traffic analysis using graph neural networks. *IEEE Trans Inf Forensics Secur* 16:2367–2380. <https://doi.org/10.1109/TIFS.2021.3050608>
 34. Lin X, Xiong G, Gou G, Li Z, Shi J, Yu J (2022) ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: Laforest F, Troncy R, Simperl E, Agarwal D, Gionis A, Herman I, Médini L (eds) WWW '22: The ACM web conference 2022, Virtual Event, Lyon, France, April 25–29, 2022, pp. 633–642. <https://doi.org/10.1145/3485447.3512217>
 35. Chen G, Peng P, Li G, Tian Y (2023) Training full spike neural networks via auxiliary accumulation pathway. <https://doi.org/10.48550/ARXIV.2301.11929>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Qin Zeng is currently working towards his PhD degree in Information and Communication Engineering at Information Engineering University, Zhengzhou, China. His current research interests include network traffic classification and artificial intelligence.



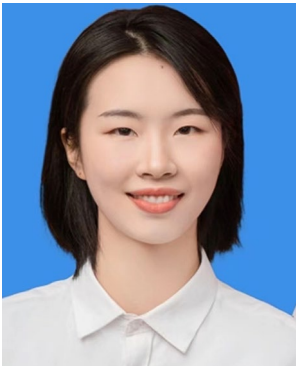
Dan Qu was born in Jilin, China, in 1974. She received the B.S., M.S. and Ph.D. degrees in information and communication engineering from the Zhengzhou Information Science and Technology Institute, Zhengzhou, China, in 2004, 2007 and 2013, respectively. From 2016 to 2017, she was a visiting scholar in Computer Science Institute of Carnegie Mellon University. She is a Professor at the College of Information Systems Engineering of Information Engineering University. Her research interests are in speech signal processing and pattern recognition & machine learning, and natural language processing.



Hao Zhang received his B.S. and Master degrees in information and communication engineering from the Information Engineering University, Zhengzhou, China, in 2019 and 2021, respectively. He is currently working towards the Ph.D. degree on speech translation at the Information System Engineering of Information Engineering University. His research interests include automatic speech translation, machine translation.



Wen-Lin Zhang received the M.S. and Ph.D. degrees in information and communication engineering from the Information Engineering University Zhengzhou, China, in 2007 and 2013, respectively. He is an Assistant Professor at the College of Information System Engineering of Information Engineering University. His research interests are in the area of speech signal processing and machine learning.



Ya-Qi Chen is currently working towards her PhD degree in Information and Communication Engineering at Information Engineering University, Zhengzhou, China. Her research interests are speech signal processing, continuous speech recognition, and meta learning.